COMP4135 SEM Weekly Report 2

Group 2 (Tactical Cards)

Team members

Guangjun HU (20511787)
Hongshuo HU (20808603)
Cheng YE (20513323)
Zehan CHEN (20807750)

School of Computer Science
University of Nottingham

Submission Date: 03/04/2026

Contents

1 Specification and Design Decisions **2**
1.1 Requirements Validation . . . 2
1.1.1 Updated Functional and Nonfunctional Priorities . . . 3
1.1.2 Key Design Trade-offs . . . 4
1.2 Architectural/System Diagram . . . 4

2 Prototyping Process **6**
2.1 Prototype . . . 6
2.2 Design Validation . . . 9

3 Implementation Strategy **11**
3.1 Code Structure . . . 11
3.1.1 Runtime Responsibilities Beyond the Diagram . . . 12
3.2 Critical Implementation Choices . . . 13

4 Summary and Reflections **15**
4.1 Team Members and the Division of Labor . . . 15
4.2 Completed Process . . . 15
4.3 Future Plan . . . 16

Reference **17**

A Meeting Minutes and Working Records **18**
A.1 Recovered Checkpoint (15/03/2026) . . . 18
A.2 WR1 Kickoff and Alignment Meetings . . . 19
A.2.1 Meeting 1 (17/03/2026) . . . 19
A.2.2 Meeting 2 (18/03/2026) . . . 19
A.3 WR2 Evidence Freeze Note . . . 20
A.4 Supporting Record Index . . . 20
A.5 Retrospective Coordination Board . . . 22

Chapter 1

Specification and Design Decisions

This chapter explains how Tactical Cards moved from the WR1 proposal to the current Unity prototype. It uses the WR1 requirements, scenarios, use cases, and role allocation as the baseline, with the related files listed in Appendix A.4. WR2 checks which WR1 requirements are now implemented, which were simplified, and which were postponed. The chapter follows the course specification and WR2 template expectations [1, 4]. It is based on the frozen WR2 snapshot recorded in the freeze note listed in Appendix A.4. As a result, it describes the submitted demo and leaves later goals outside this chapter.

1.1 Requirements Validation

WR1 defined a small tactical card battle demo: a card-driven turn loop, a bounded board, visible enemy actions, readable combat state, and enough documentation to justify implementation choices. WR2 validates that baseline against the current prototype. Some requirements now work as intended, while others were narrowed so the demo stayed playable within the coursework scope.

The current prototype validates the core loop of selecting a unit, drawing cards, playing a limited number of command cards, ending the turn, and observing an enemy response. Some presentation and completion-related requirements are still partial. The main WR1 to WR2 validation outcomes are:

- **6x6 tactical battle board.** Status: retained and implemented. Evidence: the current Unity scene and prototype screenshots. Reason: a single 6x6 scene is large enough to demonstrate positioning without expanding into campaign-scale scope.
- **Hand zone, turn controls, and combat information in one interface.** Status: retained and implemented. Evidence: the current `BattleUI` panel and screenshots. Reason: a left-side control panel plus in-world board keeps the prototype readable for reporting and demonstration.
- **Move, Strike, Guard, and Push as the starter card set.** Status: retained and narrowed. Evidence: the fallback deck and explicit card res-

olution logic. Reason: these four cards cover movement, damage, defense, and displacement with minimal rules overhead.

- **Card draw, discard, and fixed plays per turn.** Status: retained and implemented. Evidence: current turn flow and deck management logic. Reason: three cards drawn per turn and two playable actions proved easier to demonstrate than a wider energy or deckbuilding model.

- **Enemy units perform visible actions during the enemy phase.** Status: retained but simplified. Evidence: deterministic enemy movement and attack summary output. Reason: the prototype validates enemy response without requiring a broader AI system.

- **Clear visual and animation feedback for all actions.** Status: modified and partial. Evidence: tile highlights, card tinting, selection state, and status log. Reason: bespoke combat animation was reduced to lightweight prototype cues to keep WR2 scope under control.

- **Win/lose handling ends the combat session cleanly.** Status: retained and implemented in minimal form. Evidence: the current turn manager now stops the battle loop when one side is eliminated, and the battle UI surfaces a textual victory/defeat outcome. Reason: the team fixed the earlier accuracy gap with a narrow runtime change, while leaving richer result presentation for later polish.

- **Minimal onboarding cues for first interactions.** Status: partial. Evidence: the current UI gives a status prompt such as "Select a player unit," but not a fuller guided onboarding flow. This gap was confirmed during WR2 validation.

- **Structured repository workflow and traceable progress records.** Status: partially implemented and now evidenced through the cleaned GitHub workflow. Evidence: merged pull requests on the canonical repository, the collaboration-history note, and the process evidence pack listed in Appendix A.4. Reason: the repository workflow is now visible in GitHub history, but task-board captures and stronger attendance evidence are still weaker than the repository trail itself.

1.1.1 Updated Functional and Nonfunctional Priorities

Implementation kept the functional scope to one starter deck and one readable combat loop. The retained starter card set is `Move`, `Strike`, `Guard`, and `Push`. WR2 does not expand into a larger card pool or more experimental mechanics such as persistent status effects, directional bonuses, or health-for-power exchanges, because those would need more balancing and testing than the current report can support.

The main remaining non-functional issue is readability. The current build already shows the most important state in visible places: unit hit points and guard state can be checked during play, the status prompt tells the player when

to select a unit or target, and battle-end text appears in both the top bar and the combat log. However, the combat HUD is still basic. Board highlight meanings are not fully explained, and the result presentation still relies mainly on text.

1.1.2 Key Design Trade-offs

The main trade-off in WR2 was breadth versus stability. A larger deck and more specialised combat roles could add depth, but they would also increase balancing work and testing risk. The team therefore kept the card set small, kept enemy behaviour deterministic, and kept script responsibilities easy to trace. This supports the main WR2 goal: a working slice that the report can describe clearly.

The WR2 scope is smaller than the broader WR1 concept and one-pager listed in Appendix A.4. The frozen snapshot covers one battle scene, two player units, one enemy response loop, four starter cards, board-state feedback, and battle-end text feedback. Larger card pools, multi-scene flow, deckbuilding, progression, and a fuller tutorial sit outside the current implementation window described in the architecture-constraint note in Appendix A.4. They are treated as deferred work.

1.2 Architectural/System Diagram

The current architecture follows the single-battle direction set out in the architecture note and first-pass script list in Appendix A.4. Figure 1.1 shows the runtime modules used by the demo. An architecture diagram is used here because this section needs to show how scene managers, UI, cards, grid, and units interact at runtime.

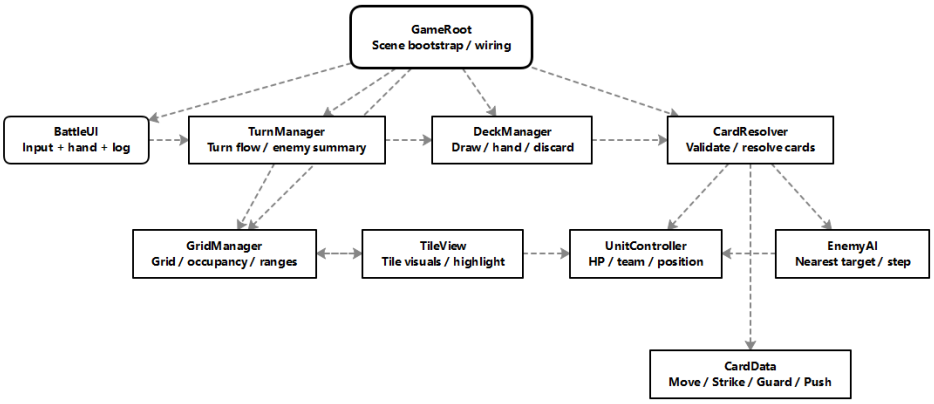

Figure 1.1: Current WR2 runtime architecture for the Tactical Cards prototype

The control flow begins at `GameRoot`, which wires together the scene managers and registers the units already placed in the scene. `TurnManager` owns turn state, remaining card plays, and the switch between player and enemy

phases. `BattleUI` stays small: it handles card selection, world clicks, and status output, then passes gameplay decisions to the deck, grid, and card systems.

The data flow is simple. `DeckManager` owns draw, hand, and discard state. `CardResolver` is the only module that interprets `CardType`, which keeps card behaviour explicit. `GridManager` owns tile generation, occupancy, and reachability, while `UnitController` stores runtime unit state such as hit points, team, guarding, and grid position. This matches the current prototype goal: one readable vertical slice.

Chapter 2

Prototyping Process

The prototype moved through three stages: an early concept note (`onepager_en.md`), an architecture-constraint note (`demo_architecture_constraints.md`), and a first-pass script list (`demo_first_pass_scripts.md`), all indexed in Appendix A.4. This matches the prototype-first approach described in the WR2 teaching material [2]. WR2 records the point where those design decisions became a playable vertical slice. This chapter describes the frozen WR2 snapshot, so deferred items stay labelled as deferred.

2.1 Prototype

Figure 2.1 shows a retrospective low-fidelity wireframe rebuilt from the WR1 scope and the current scene layout. It is marked as retrospective because no separate low-fidelity mockup was archived before prototype implementation began. The wireframe is still useful because it clarifies the intended information zones: turn state, hand controls, status feedback, and the board.

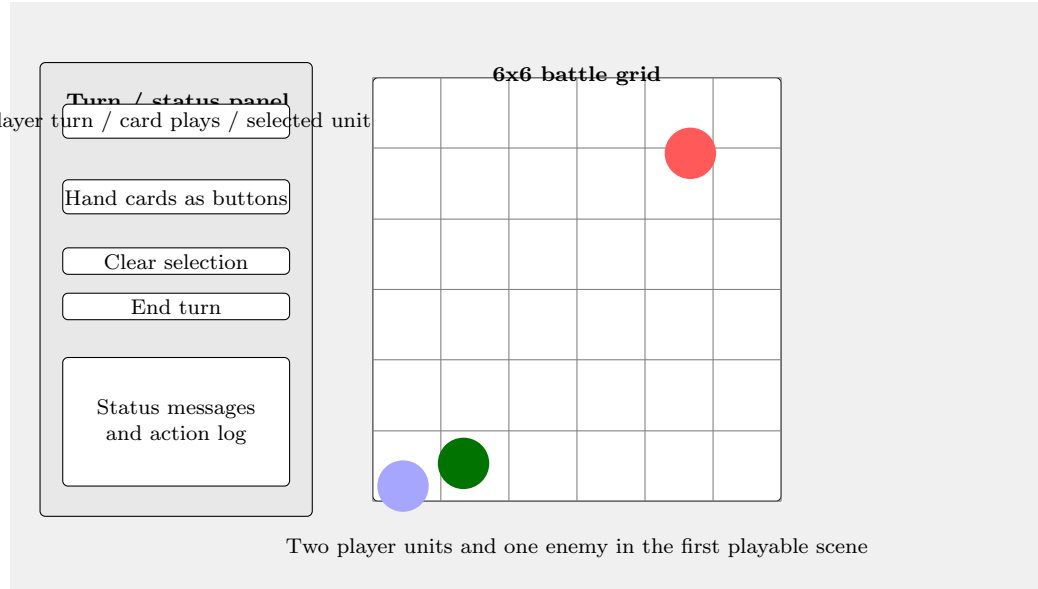

Figure 2.1: Retrospective low-fidelity wireframe reconstructed from WR1 scope and the current prototype layout

The high-fidelity prototype evidence comes from the working Unity scene. Figure 2.2 shows the current battle board and left-side battle UI, Figure 2.3 focuses on the playable card hand and top-bar turn feedback, and Figure 2.4 isolates the in-world battle scene used as the basis for interaction and screenshot capture.

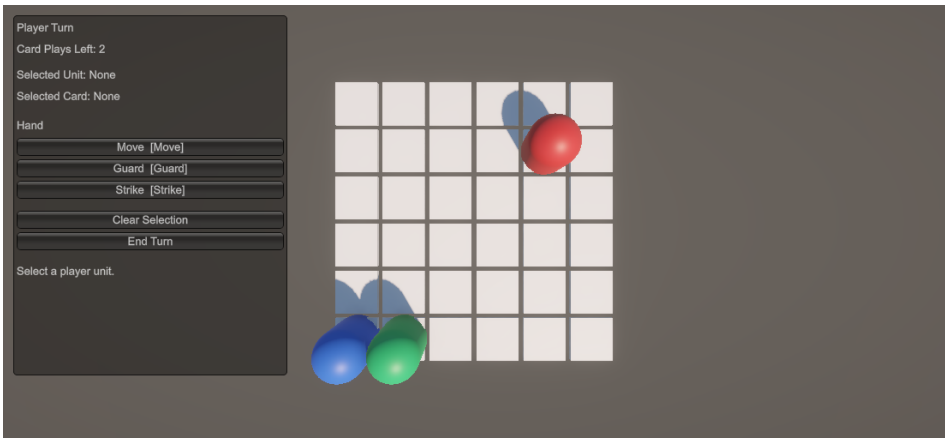

Figure 2.2: Current high-fidelity prototype with battle UI, starter hand, and 6x6 board

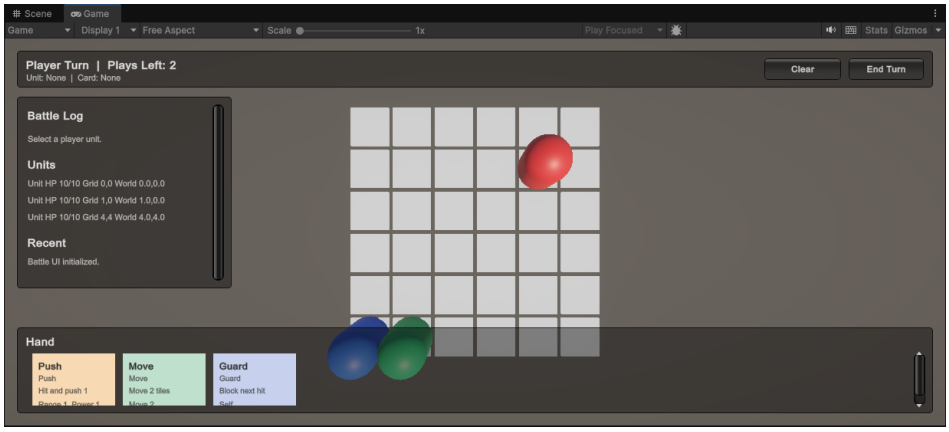

Figure 2.3: Playable hand cards and turn-state UI in the current prototype scene

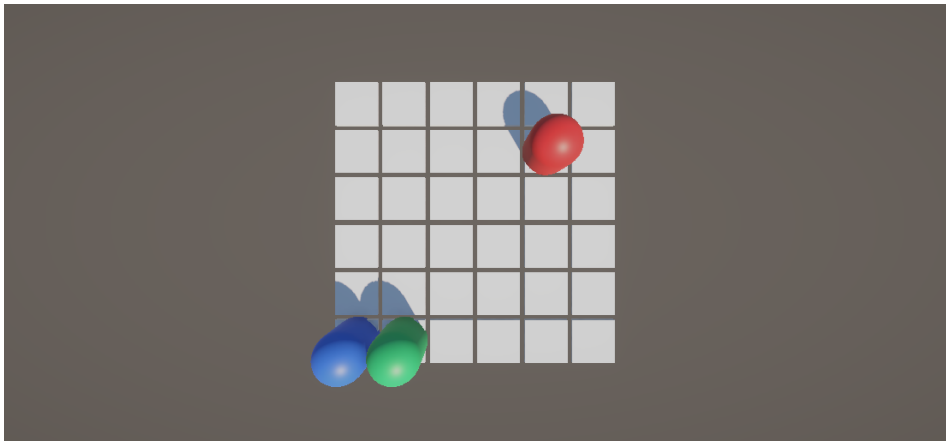

Figure 2.4: Board-only view of the current playable battle scene

The current frozen prototype feature state can be summarised as follows:

- **Implemented.** 6x6 battle grid, unit selection, starter hand and card draw, Move/Strike/Guard/Push resolution, enemy response in simplified form, board highlight feedback, and battle-end gating with textual victory/defeat reporting.
- **Partial.** Onboarding cues and feedback emphasis. The UI prompts the player. It does not yet teach the full interaction sequence or explain all board highlight meanings. The battle-end result is visible, but still lightweight.
- **Initial automated verification.** A committed EditMode test suite checks three battle-end scenarios in the turn manager, giving WR2 a minimal automated validation layer.
- **Deferred.** Broader automated coverage, stronger external playtesting, and richer audiovisual result presentation remain for the next reporting cycle.

Several implementation constraints shaped the prototype:

- The UI is a hybrid prototype solution. It favours quick wiring and readable reporting over long-term extensibility.
- Input uses the current Input System plus physics raycasts. This links button-based card choice to tile and unit selection with little extra infrastructure.
- Enemy behaviour is deterministic. This keeps debugging, validation, and reporting manageable.
- The prototype focuses first on the battle loop and feedback model. Later work can improve battle-end presentation, onboarding, and automated coverage.

2.2 Design Validation

WR2 validation used an internal heuristic walkthrough of the frozen prototype snapshot on 01/04/2026. The local workspace does not yet preserve a separate user-test archive, so this stage uses structured scenario review. That matches the seminar distinction between early validation and the fuller testing structure expected in WR3 [3]. The walkthrough checked three scenarios aligned with the current demo goals: selecting a player unit, playing a card against a valid target, and ending the turn to observe enemy behaviour. The note is listed in Appendix A.4 as `validation/wr2-heuristic-walkthrough-2026-04-01.md`. After that walkthrough, one narrow runtime fix and an EditMode test pass were added to correct the battle-end accuracy gap without widening the prototype scope.

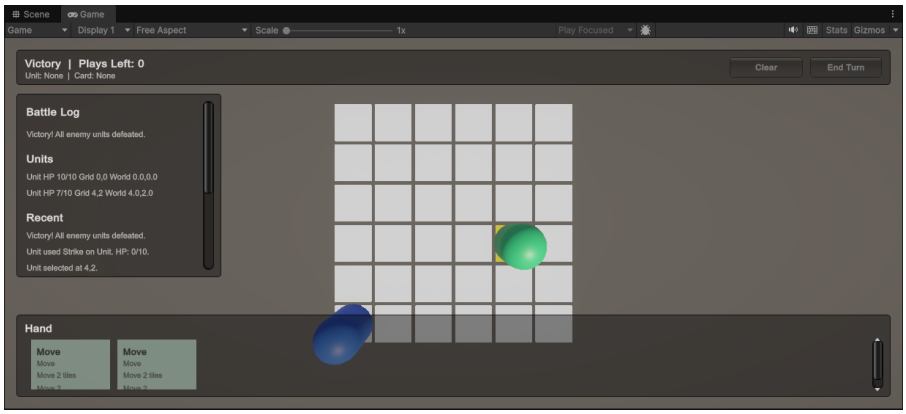

Figure 2.5: EditMode smoke test result recorded after the battle-end accuracy fix

Figure 2.5 adds an automated check alongside the walkthrough evidence. It does not replace broader WR3 testing. It does show that the corrected battle-end behaviour was executed successfully in the Unity Test Framework.

The key validation findings are:

1. **Scenario: first interaction and unit selection.** Finding: the player can infer the next step from the left-side status prompt, but there is no

stronger onboarding cue on the board itself. Severity: medium. Proposed action: add a clearer first-turn prompt or highlight the expected first click. Status: planned.

2. **Scenario: play a card and confirm its target.** Finding: card meanings are readable, but the board does not explain move versus attack highlight colours directly. Severity: medium. Proposed action: add a tiny legend or one-line prompt when a card is selected. Status: planned.

3. **Scenario: end turn and observe enemy response.** Finding: enemy actions were readable through summary text, but the battle loop initially lacked a clear end-state gate and outcome cue. Severity: high. Proposed action: add explicit battle-end termination and visible victory/defeat feedback before final WR2 closeout. Status: applied in narrow form through the prototype-accuracy fix; stronger post-action emphasis remains planned.

These findings shaped the final wording of WR2. The report claims a working vertical slice with battle-end gating, text outcome feedback, and an initial automated check. It does not claim a finished tutorial flow, a polished result screen, or full audiovisual combat feedback.

Chapter 3

Implementation Strategy

The implementation strategy follows one principle: keep the prototype readable enough that each major gameplay responsibility can be traced to a small number of Unity scripts. This section focuses on code structure and the main trade-offs behind the current prototype.

3.1 Code Structure

At repository level, the project is split so that the Unity implementation, report sources, and evidence records do not collapse into one mixed file tree. The simplified WR2-relevant top-level layout used during report closeout is:

```
Unity-Project
|-- coursework
|-- artifacts/uml
`-- doc/src
    |-- project-records
    |-- wr1-latex
    `-- wr2-latex
```

This layout keeps the playable Unity project inside `coursework` and stores diagrams and report evidence separately. Within that structure, the current gameplay scripts are organised into a small set of folders under `coursework/Assets/Scripts`. The directory tree below reflects the runtime structure used by the prototype.

```
Assets/Scripts
|-- Cards
|   |-- CardData.cs
|   |-- CardResolver.cs
|   `-- DeckManager.cs
|-- Common
|   `-- GameEnums.cs
|-- Core
|   |-- GameRoot.cs
|   `-- TurnManager.cs
|-- Grid
|   |-- GridManager.cs
```

```
|   `-- TileView.cs
|-- UI
|   |-- BattleUI.cs
|   `-- CardButtonView.cs
`-- Units
    |-- EnemyAI.cs
    `-- UnitController.cs
```

This structure mirrors current responsibilities. The `Core` folder owns scene bootstrapping and turn flow. `Grid` owns tile generation, occupancy, and range queries. `Cards` owns card data, deck state, and explicit card resolution. `Units` owns health, team, and enemy decision logic. `UI` connects player input to runtime state.

A file-tree view is used here because this prototype is built around a small set of scene-bound Unity `MonoBehaviour` scripts. The clearest way to communicate structure is to show where gameplay responsibilities live under `Assets/Scripts`. A more traditional inheritance-heavy class diagram would suggest architectural depth that the current prototype does not actually have.

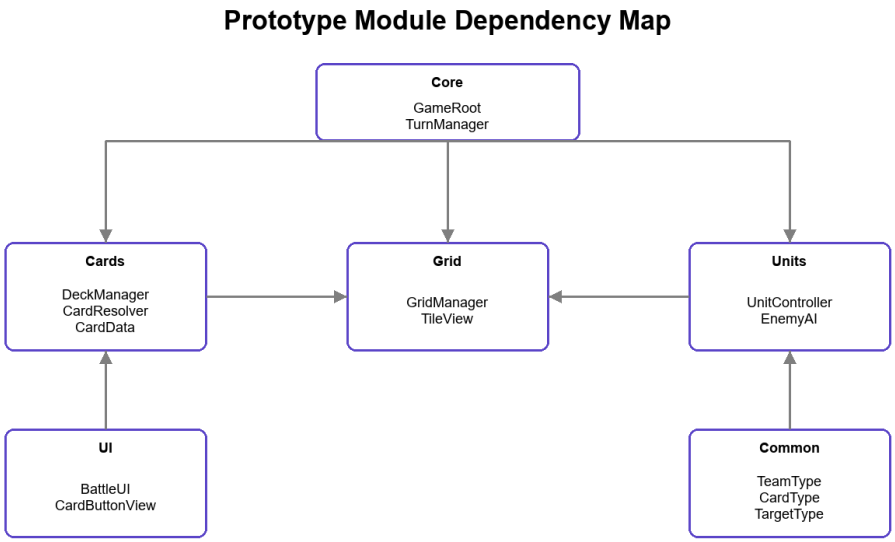

Figure 3.1: Simplified module dependency and responsibility map

Figure 3.1 explains code responsibility split instead of runtime event order. Dependencies flow inward toward a small number of managers instead of outward into a large inheritance tree. This matches the goal of a small, traceable prototype.

3.1.1 Runtime Responsibilities Beyond the Diagram

The responsibility map is simplified, so the feature-level role of each main script is summarised below:

- `GameRoot`. Registers the pre-placed units in the scene, links the managers together, and provides the starting point for the playable battle slice.

- `TurnManager`. Controls player and enemy phase transitions, remaining card plays, start-of-turn draw behaviour, and the battle-end gate introduced by the prototype-accuracy fix.
- `GridManager`. Builds the 6x6 board, tracks tile occupancy, and answers movement or target-range queries used by cards and AI.
- `DeckManager`. Owns the starter deck, draw pile, discard pile, and current hand shown in the battle UI.
- `CardResolver`. Converts `Move`, `Strike`, `Guard`, and `Push` into explicit runtime effects so card behaviour stays easy to trace and test.
- `UnitController`. Stores team, hit points, guarding state, and grid position, and exposes the small set of combat operations needed by the current prototype.
- `EnemyAI`. Produces the deterministic enemy response used in WR2, keeping target choice and movement logic simple enough to observe clearly during demonstration.
- `BattleUI` **and** `CardButtonView`. Handle card selection, click routing, prompts, and log updates so the player can see the state changes caused by the core systems above.

3.2 Critical Implementation Choices

The most important implementation choices in the current prototype are:

- **Single self-contained battle.** Options considered: one battle scene versus multi-scene campaign flow. Chosen option: one battle scene, because it provides a complete vertical slice for WR2 without spending time on navigation and persistence. Trade-off: broader progression is postponed.
- **Shared unit runtime model.** Options considered: one shared controller versus separate player/enemy hierarchies. Chosen option: one shared `UnitController` so that health, guarding, movement, and grid state remain easy to trace. Trade-off: the model is less specialised than a longer-term production structure.
- **Explicit card resolver.** Options considered: `enum`-driven explicit logic versus a generic card-effect hierarchy. Chosen option: one resolver that interprets `CardType`, because the starter set is small and benefits from readable switch-based logic. Trade-off: a larger future card set would eventually need refactoring.
- **Deterministic enemy AI.** Options considered: nearest-target movement versus broader tactical AI. Chosen option: deterministic nearest-target behaviour, because it is predictable, testable, and easy to explain in WR2. Trade-off: the enemy behaviour remains strategically shallow.

- **Prototype-first UI and input path.** Options considered: a full Unity UI pipeline versus the current hybrid prototype UI with Input System and board raycasts. Chosen option: the hybrid path, because it was faster to wire into the existing scene and sufficient for WR2 reporting. Trade-off: UI consistency and polish remain limited.

- **GitHub-first collaboration after repository cleanup.** Options considered: continue closing WR2 on a mixed local clone versus migrate report-closeout work onto a cleaned GitHub workflow with explicit PR rules. Chosen option: move current collaboration onto the cleaned GitHub repository, record the review policy, and keep earlier local/Overleaf/WeChat-era activity in supporting notes. Trade-off: current closeout work is now traceable through merged pull requests, but some earlier process evidence still relies on reconstructed notes and the task-board trail remains weaker than the repository history.

The most important strategic choice was to treat the prototype as implementation evidence for a small tactics slice. That choice explains why the runtime modules are explicit, why the card system stays narrow, and why some features are marked as deferred. For WR2, this keeps the implementation aligned with the report and the current demo scope.

Chapter 4

Summary and Reflections

4.1 Team Members and the Division of Labor

The implementation-phase allocation used in WR2 preparation is summarised below. It updates the earlier WR1 proposal roles and reflects the work packages needed to turn the concept into a working prototype and report pack.

- Guangjun HU (20511787): WR2 integration, requirements validation, appendix consolidation, and document traceability.
- Hongshuo HU (20808603): Unity architecture, scene wiring, core battle systems, and runtime integration.
- Cheng YE (20513323): UML and figure refinement, prototype presentation material, and supporting chapter drafting.
- Zehan CHEN (20807750): validation records, testing and meeting templates, and supporting documentation/website tasks.

4.2 Completed Process

By the end of this WR2 cycle, the project has moved from a proposal-level concept to a working prototype snapshot. Building on the WR1 requirement baseline, the team produced a constrained architecture definition, implemented the first-pass gameplay modules, captured prototype screenshots, updated the runtime architecture to match the current code, and aligned the report with the current prototype evidence. During closeout, the team also corrected a battle-end accuracy gap by adding battle-end termination and textual outcome feedback, then backed that fix with a small EditMode test suite around the turn manager.

This reporting cycle also exposed a process gap: several implementation records were not yet archived in a reusable structure. WR2 therefore introduces a small project-record set covering meeting notes, process evidence notes, decision logs, validation notes, and testing records. The cleaned GitHub repository now also shows the current closeout workflow through merged document

and code pull requests. These records are lightweight so the team can continue using them in WR3 and the final report.

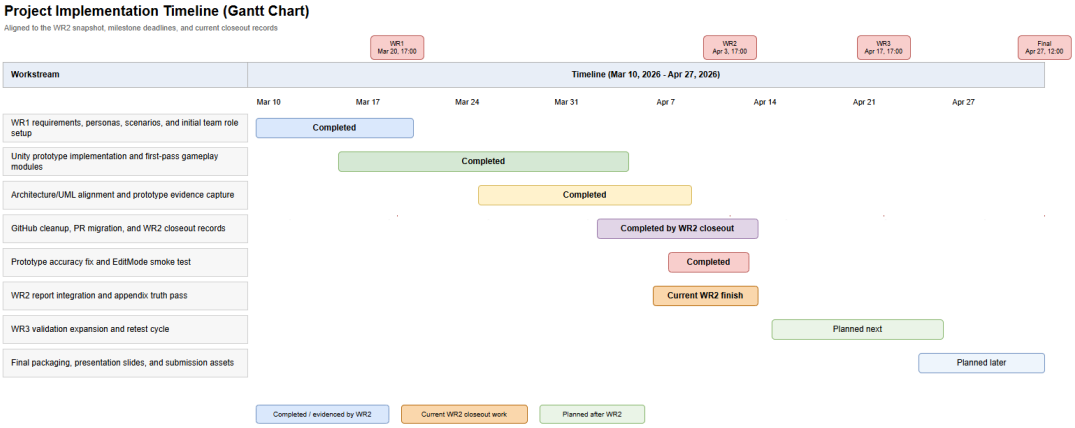

Figure 4.1: Implementation timeline aligned to coursework milestones and the current WR2 closeout snapshot

Figure 4.1 summarises the timeline from the WR1 foundation through the current WR2 closeout and the planned WR3/final-report stages. The chart separates work already completed in WR2 from work scheduled after the WR2 deadline.

4.3 Future Plan

The next stage connects directly to WR3 and the final report:

- strengthen battle-end presentation beyond the current top-bar/log feedback and tighten enemy-turn emphasis;
- improve onboarding cues for first-time interaction with the board and card selection flow;
- supplement the now-active GitHub PR history with any stronger task-board captures or confirmed attendance evidence the team can still recover;
- convert the current validation notes into a fuller WR3 testing pack with test cases, bug tracking, and retest records;
- expand the current automated coverage beyond battle-end behaviour into stable rule-heavy systems such as grid range checks and card resolution rules.

In short, WR2 has produced a stable prototype foundation and a clearer list of follow-up work for WR3.

Bibliography

[1] University of Nottingham. Comp4135 coursework 1 specification, 2026. Local course specification used for WR2 scope, deadlines, and weekly report expectations.

[2] University of Nottingham. Comp4135 seminar 5: Prototyping, 2026. Seminar material on prototype goals, fidelity, and requirement validation.

[3] University of Nottingham. Comp4135 seminar 6: Software validation, 2026. Seminar material on validation, implementation design, and testing preparation.

[4] University of Nottingham. Comp4135 weekly report 2 template, 2026. Moodle WR2 template PDF used to recover the correct Weekly Report 2 chapter structure.

Appendix A

Meeting Minutes and Working Records

A.1 Recovered Checkpoint (15/03/2026)

No formal implementation meeting minutes were archived in the local workspace for the first prototype checkpoint. This recovered note summarises the result of an early classroom discussion and follow-up implementation alignment that informed the first prototype slice, but that in-class discussion itself was not formally recorded. To avoid inventing details, the following checkpoint summary was therefore compiled from dated artifacts that are present in the repository: the architecture constraints note, the first-pass script plan, and the prototype screenshots created on 15/03/2026.

- **Record type:** recovered implementation checkpoint compiled from local artifacts after an unrecorded classroom discussion, rather than from formal minutes.
- **Date:** 15/03/2026.
- **Attendance:** exact attendance was not archived in the local workspace and should be replaced with confirmed team attendance if chat or calendar records are available.
- **Sources used:** the architecture note and first-pass script plan listed later in the Supporting Record Index, plus prototype screenshots timestamped on 15/03/2026.
- **Main focus:** reduce the original game concept to one battle scene, four starter cards, a readable control loop, and a minimum set of scripts that can compile and run inside the existing Unity project.
- **Confirmed decisions:** single-battle scope, shared unit controller, deterministic enemy response, explicit card types, button-driven card interaction, and no campaign or deckbuilding systems in WR2.
- **Follow-up actions:** finish the first gameplay skeleton, refresh UML diagrams to match implementation, and gather prototype evidence for WR2.

A.2 WR1 Kickoff and Alignment Meetings

The following two short records are retained because WR2 directly inherits the role allocation and initial scope decisions set during WR1 preparation.

A.2.1 Meeting 1 (17/03/2026)

Time

17/03/2026

Attendance

- Guangjun HU
- Hongshuo HU
- Zehan CHEN

Context

Team members shared their initial game ideas and implementation interests, then narrowed the coursework direction to a tactical card battle demo. The meeting also established the first division of labour and the expected delivery dates for WR1 inputs.

Decisions

Project direction was confirmed as Tactical Cards, the initial role allocation was agreed, and individual task expectations were set for the first weekly cycle.

Owner

Guangjun HU, Hongshuo HU, and Zehan CHEN, with follow-up input from Cheng YE.

Deadline

18/03/2026 to 19/03/2026 for the first WR1 materials and task allocation outputs.

Follow-up

These decisions fed directly into the project direction, division of labour, and requirement sections of WR1.

A.2.2 Meeting 2 (18/03/2026)

Time

18/03/2026

Attendance

- Guangjun HU
- Cheng YE

Context

The team aligned the current requirement scope around a tactical card battle demo, confirmed UML-related responsibilities, and standardised a lightweight meeting-minutes format so later weekly records would be more traceable than the earliest prototype cycle.

Decisions

The current WR1 scope was aligned around a tactical card battle demo, UML diagram production was assigned, and a unified meeting-minutes format was defined for subsequent meetings.

Owner

Guangjun HU and Cheng YE, with follow-up support from the whole team.

Deadline

19/03/2026 for WR1 alignment, followed by use of the agreed meeting-minutes format in later weekly cycles.

Follow-up

These outcomes were reflected in WR1 requirements, UML diagrams, and the documentation process used by later reports.

A.3 WR2 Evidence Freeze Note

The WR2 documentation pack uses an evidence freeze line of **02/04/2026 12:00**. This freeze is recorded in `wr2-snapshot-freeze-2026-04-02.md`. After that point, only changes that materially improve submission quality should be described in WR2. New features developed later should be documented in WR3 or the final report instead of being backfilled into this report.

A.4 Supporting Record Index

The file set referenced across WR2 is summarised below. It combines upstream design files under `doc/src` with later supporting records under `doc/src/project-records`. Internal file mentions in the main report point back to this index so the referenced materials can still be located clearly from the repository layout.

- `doc/src/wr1-latex/main.tex`: WR1 report source containing the original requirement, scenario, and system-model baseline inherited by WR2.
- `doc/src/onepager_en.md`: early internal concept note used before the prototype scope was narrowed.
- `doc/src/demo_architecture_constraints.md`: internal architecture-constraint note used to cut the concept down to one battle scene.
- `doc/src/demo_first_pass_scripts.md`: first-pass gameplay script plan for turning the narrowed design into a playable Unity slice.
- `doc/src/project-records/wr2-snapshot-freeze-2026-04-02.md`: freeze note defining the prototype and evidence snapshot used for WR2.
- `wr2-asset-register.md`: reusable evidence list, missing items, suggested owners, and due times.
- `process/canonical-repo-evidence-2026-04-02.md`: canonical repository evidence note, historical discovery summary, and the current recommendation for using merged GitHub history as repo-workflow evidence.

- `process/task-board-evidence-2026-04-02.md`: task-board evidence note, discovery summary, and import checklist.
- `process/project-implementation-timeline-gantt.drawio`: draw.io source for the coursework timeline Gantt chart used in the WR2 reflections section.
- `process/task-board/retrospective-wr2-closeout-board.png`: retrospective coordination board reconstructed from the recorded TODO list and merged PR history for explanation only.
- `process/discovery/README.md`: command-level discovery artifacts explaining what was checked from this workspace.
- `meetings/meeting-minutes-template.md`: lightweight meeting template for future weekly records.
- `meetings/meeting-2026-03-17-wr1-kickoff.md`: preserved WR1 kickoff note covering project direction and first role allocation.
- `meetings/meeting-2026-03-18-wr1-alignment.md`: preserved WR1 alignment note covering scope confirmation and meeting-template setup.
- `meetings/implementation-checkpoint-recovered-2026-03-15.md`: transparent recovered note based on local artifacts.
- `decisions/wr2-implementation-decisions.md`: decision log extracted from the current implementation direction.
- `collaboration-history-2026-04-02.md`: concise explanation of the move from early WeChat/Overleaf coordination to the current GitHub PR workflow.
- `validation/wr2-heuristic-walkthrough-2026-04-01.md`: internal WR2 walkthrough and findings.
- `testing/prototype-accuracy-editmode-run-2026-04-02.md`: EditMode verification note for the battle-end accuracy fix.
- `testing/wr3-prep-log-2026-04-01.md`: initial test and bug tracking preparation note for the next report cycle.

These records are intentionally lightweight. Their purpose is to stop evidence loss between WR2 and WR3, not to pretend that the record-keeping process was already complete earlier in the prototype cycle.

A.5 Retrospective Coordination Board

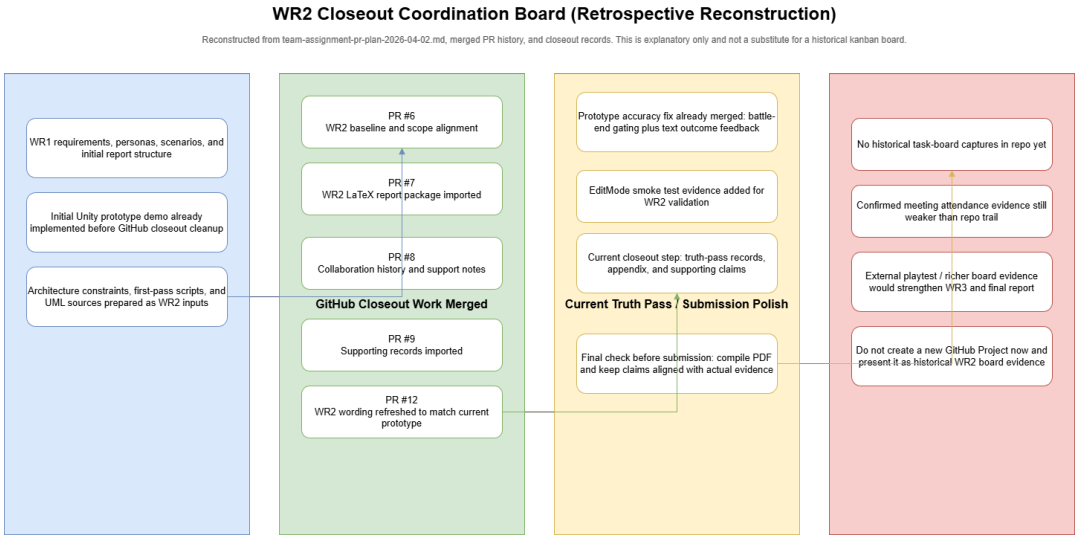

Figure A.1: Retrospective reconstruction of the WR2 closeout board, based on recorded TODO lists and merged GitHub PR history

Figure A.1 is included as an explanatory reconstruction because no historical task-board capture survives in the repository. It helps summarise how the closeout work was grouped after the move to GitHub PR workflow, but it is not claimed as original planning-board evidence from the active implementation period.